Integration of Kinematics and Reinforcement Learning for Hexapod Locomotion

Muhammad Harish[1],∗, Muhammad Abdillah[1], Novanto Yudistira[2]

[1] Department of Electrical Engineering, Faculty of Industrial Technology, Universitas Pertamina, Jakarta, Indonesia
[2] Department of Informatics, Faculty of Computer Science, Brawijaya University, Malang, Indonesia
∗ Corresponding author: 102121014@student.universitaspertamina.ac.id

Abstract

This paper presents the implementation of Reinforcement Learning (RL) on a hexapod robot to enhance its adaptability in complex terrains. The hexapod robot, named BumbleBee, integrates kinematic methods (inverse kinematics, body kinematics, and trajectory planning) with RL algorithms to achieve stable and efficient locomotion. Three RL algorithms Proximal Policy Optimization (PPO), Twin Delayed Deep Deterministic Policy Gradient (TD3), and Soft Actor-Critic (SAC) were trained in a PyBullet simulation. SAC demonstrated superior performance with a cumulative reward of -1300, outperforming TD3 (-2200) and PPO (-4400). The trained SAC model was deployed on the physical BumbleBee robot and tested on real-world terrains, including slopes and stairs. Results showed a 100% success rate, with body stability maintained within ±0.3 radians. This study highlights the effectiveness of combining RL with kinematic control for adaptive hexapod locomotion in unstructured environments.

Keywords: Hexapod robot, Reinforcement Learning, Soft Actor-Critic, inverse kinematics, adaptive control

1. Introduction

Hexapod robots, inspired by insects, are designed for stable locomotion across unstructured terrains due to their six-legged configuration. These robots are particularly useful in search and rescue (SAR) missions, where navigating complex environments is critical. However, the high degrees of freedom (18 DoF for a 3-DoF per leg hexapod) introduce significant control challenges, making traditional manual control methods inefficient and unstable [1], [2].

Recent advancements in robotics have explored the integration of kinematic methods with machine learning to address these challenges. Inverse kinematics (IK) and body kinematics (BK) provide precise control over leg movements and body stability, while trajectory planning ensures dynamic motion [3], [4] . However, these methods lack adaptability to unpredictable terrain changes. Reinforcement Learning (RL) offers a promising solution by enabling robots to

learn optimal movement strategies through trial and error, adapting to environmental feedback [5], [6].

This study focuses on integrating RL with kinematic control to enhance the adaptability and stability of a hexapod robot. Three state-of-the-art RL algorithms PPO, TD3, and SAC were evaluated in simulation and real-world tests. The primary contributions of this work are:

1. Development of a hybrid control system combining RL and kinematics for hexapod locomotion.
2. Comparative analysis of PPO, TD3, and SAC in achieving stable and adaptive gait patterns.
3. Successful deployment of the SAC model on a physical robot, demonstrating 100% success rate in navigating slopes and stairs.

The remainder of this paper is organized as follows: Section II reviews related work, Section III describes the methodology, Section IV presents the results, and Section V concludes the paper.

2. Related Work

The control of hexapod robots has evolved significantly, progressing from manual, pre-programmed methods to sophisticated learning-based frameworks. Early approaches relied on direct joint-level control, which proved to be unstable and inefficient due to the high-dimensional nature of the 18-DoF system [3], [4].

To overcome these limitations, researchers adopted classical kinematic models. These models form the foundation of modern hexapod locomotion, typically including:

- Inverse Kinematics (IK): This technique translates the desired Cartesian coordinates of the foot-tips (end-effectors) into the required angular positions for each joint (coxa, femur, and tibia) [5]. This simplifies control by abstracting away the individual joint movements.
- Body Kinematics (BK): This model governs the position and orientation of the robot's main body relative to its feet, ensuring that the center of mass remains within the support polygon for static stability, especially on uneven terrain [6].
- Gait and Trajectory Planning: Gait planning defines the sequence and timing of leg movements, with common patterns including the tripod gait for speed and the wave gait for stability [7]. Trajectory planning generates smooth paths for the feet during swing and stance phases, minimizing jerky movements and improving efficiency [8].

While kinematic-based controllers offer precision, they are inherently rigid. They depend on hand-tuned parameters and pre-defined gaits, which lack the adaptability required for autonomous navigation in dynamically changing environments. This has motivated the integration of machine learning, particularly Reinforcement Learning (RL), to enable adaptive locomotion.

Recent studies have demonstrated the potential of RL to optimize hexapod gaits. Ouyang et al. [4] and Li et al. [9] showed that RL agents could learn stable and energy-efficient locomotion policies in simulation. By defining a reward function that penalizes instability and high energy consumption, the agent can discover effective movement strategies through trial and error. Ji et al. [10] and

Yang et al. [11] further explored the use of RL with Central Pattern Generators (CPGs) to generate terrain-adaptive gaits, achieving robust locomotion in simulated challenging terrains.

However, a significant gap remains in the literature. Most RL-based hexapod control systems have been confined to simulation, with few successfully bridging the sim-to-real gap. Furthermore, controlling all 18 joints directly with RL results in an excessively large action space, which complicates and slows down the training process. This paper addresses these challenges by proposing a hierarchical control architecture that integrates a low-level kinematic controller with a high-level RL policy. This approach reduces the complexity of the learning problem by allowing the RL agent to operate in a low-dimensional, task-relevant action space (e.g., commanding forward velocity and body posture) while the kinematic layer handles the precise joint-level execution.

3. Metodology

The proposed system employs a hierarchical control architecture to achieve adaptive locomotion, as illustrated in Fig. 1. The framework is divided into a low-level kinematic control layer and a high-level RL policy layer. The robot platform, simulation environment, and RL formulation are detailed below.

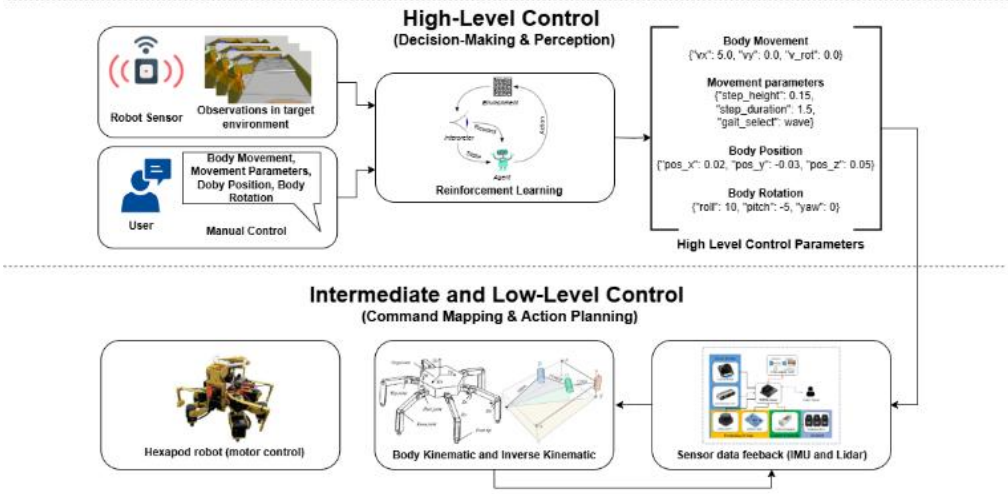


Figure 1: Hierarchical control architecture, separating high-level decision-making (RL) from low-level motion execution (Kinematics).

A. Hardware Platform

The physical experiments were conducted on a custom-built hexapod robot named "BumbleBee" (Fig. 2). The robot's chassis is constructed from 3D-printed Polylactic Acid (PLA) to ensure a lightweight yet durable frame. Each of the six legs is actuated by three Dynamixel smart servos: an AX-18A for the high-torque coxa and femur joints and an AX-12A for the tibia joint. Onboard computation is managed by an NVIDIA Jetson Nano, which is responsible for running the RL inference model and the kinematic calculations. A BNO055 Inertial Measurement Unit (IMU) provides real-time data on the robot's body orientation (roll and pitch), which is a critical input for the control policy.

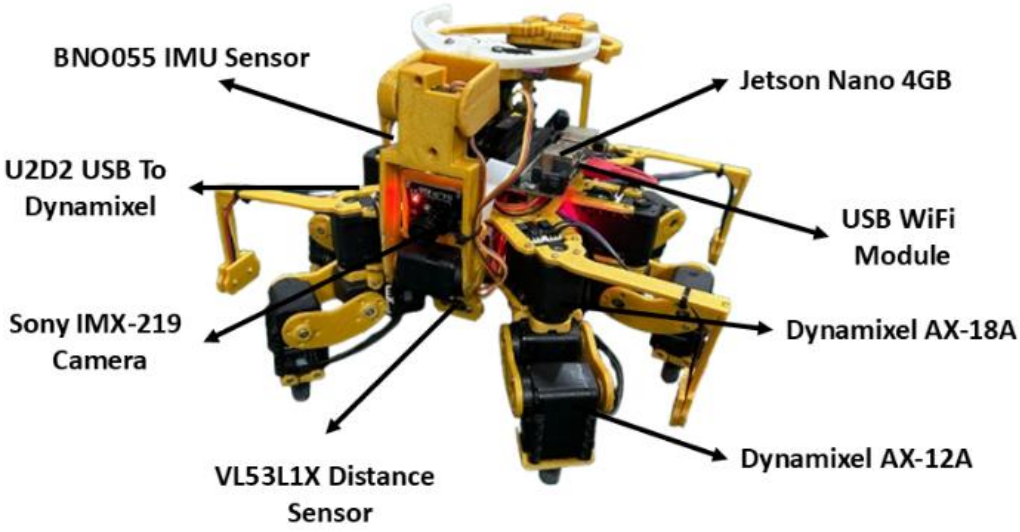


Figure 2: The "BumbleBee" hexapod robot platform used for physical validation.

B. Kinematic Control Layer

The low-level control layer is responsible for translating high-level action commands generated by the reinforcement learning (RL) policy into precise joint-level actuation. This is achieved through the implementation of standard kinematic models, enabling accurate and stable locomotion.

1. Body Kinematics:
   This module adjusts the robot's body position *(x,y,z)* and orientation *(roll,pitch,yaw)*, *(roll,pitch,yaw)* based on the high-level action vector produced by the RL agent. These transformations establish a global reference frame for subsequent gait and leg motion planning.
2. Gait Planning:
   A tripod gait is employed to maintain a balance between speed and stability. The gait cycle is phase-controlled to ensure smooth and coordinated transitions between stance and swing phases for each leg group. This pattern facilitates efficient locomotion over a variety of terrains.
3. Inverse Kinematics (IK):
   For each leg, inverse kinematics is used to compute the joint angles required to position the leg's end-effector at the desired target location defined by the body kinematics and gait planning module. Specifically, the joint angles θ_1(coxa), θ_2 (femur), and θ_3 (tibia) are calculated based on a geometric model of the leg. The IK formulation ensures that each leg moves precisely and synchronously with the body and gait dynamics.

This hierarchical structure abstracts complex locomotion dynamics, allowing the RL policy to operate at a higher level of abstraction. As a result, the agent does not need to learn the intricate low-level kinematic relationships from scratch, thereby simplifying the learning task and improving training efficiency.

C. Reinforcement Learning Framework

To learn adaptive behaviors, the control problem is formulated as a Markov Decision Process (MDP). The agent learns a policy π(at|st) that maps states to actions to maximize the expected cumulative reward. The training was conducted in a PyBullet physics simulation environment, using the Stable-Baselines3 library.

The RL agent aims to learn a policy $\pi(a_t \mid s_t)$ that maps the observation s_t to an action a_t at each timestep t, with the goal of maximizing the expected cumulative reward. Training is conducted in a physics-based simulation environment (PyBullet) using three popular RL algorithms: Proximal Policy Optimization (PPO), Soft Actor-Critic (SAC), and Twin Delayed Deep Deterministic Policy Gradient (TD3).

The state s_t provides essential information about the robot's current physical condition. In this work, the observation vector is defined as follows:

Table 1: Design Observation Space (State)

No	Observation Parameter	Range	Description
1	roll	-1.54 to 1.54 rad	Rotation around the X-axis (from IMU)
2	pitch	-1.54 to 1.54 rad	Rotation around the Y-axis (from IMU)
3	terrain_type	0 to 20	Discrete class representing ground type

The action a_t is a low-dimensional, continuous vector containing high-level motion commands, which are interpreted by the low-level kinematic controller:

Table 2: Design Action Space

No	Action Parameter	Range	Description
1	Δx	-1 to 1	Forward/backward movement (X-axis)
2	Δy	-1 to 1	Lateral movement (Y-axis)
3	Δroll	-1 to 1	Roll orientation adjustment
4	Δpitch	-1 to 1	Pitch orientation adjustment
5	step_height	0.03 to 0.1 m	Foot lift height for obstacle clearance

The reward function is designed to encourage stable, efficient, and purposeful movement. The total reward at time t, R_t, is computed as a weighted sum of three penalty-based components:

$$R_t = w_1 \cdot R_{\text{stability}} + w_2 \cdot R_{\text{energy}} + w_3 \cdot R_{\text{stuck}} \quad (1)$$

Table 3: Design Action Space

No	Reward Component	Weight	Description
1	stability_penalty	w_1	Penalty for body instability (based on roll and pitch)
2	energy_penalty	w_2	Penalty based on estimated energy usage (e.g., motor torque × velocity)
3	stuck_penalty	w_3	Penalty when the robot is stuck and not making progress for a duration

The terrain_type used in the observation space is estimated from the robot's orientation using IMU data. The RL agent implicitly learns terrain characteristics by observing roll and pitch fluctuations. In future work, additional perception modules such as visual or depth sensors could be integrated for explicit terrain recognition.

Table 4: Design Action Space

Terrain Type	Threshold Roll (°)	Threshold Pitch (°)
0	$0 < \text{roll} < 5$	$0 < \text{pitch} < 5$
1	$5 \leq \text{roll} < 10$	$5 \leq \text{pitch} < 10$
2	$10 \leq \text{roll} < 15$	$10 \leq \text{pitch} < 15$
3	$15 \leq \text{roll} < 20$	$15 \leq \text{pitch} < 20$
4	$20 \leq \text{roll} < 25$	$20 \leq \text{pitch} < 25$
5	$\text{roll} \geq 25$	$\text{pitch} \geq 25$

4. Result and Discussion

The performance of the proposed framework was rigorously evaluated through a two-stage process: (1) a comparative analysis of the PPO, SAC, and TD3 algorithms in a high-fidelity simulation, and (2) the deployment and validation of the best-performing model on the physical "BumbleBee" hexapod across real-world terrains.

A. Simulation-Based Algorithm Comparison

The primary objective of the simulation was to identify the most suitable RL algorithm for achieving stable and adaptive locomotion. The algorithms were trained for 100,000-time steps, and their performance was evaluated based on training stability, final policy effectiveness, and robustness to environmental challenges.

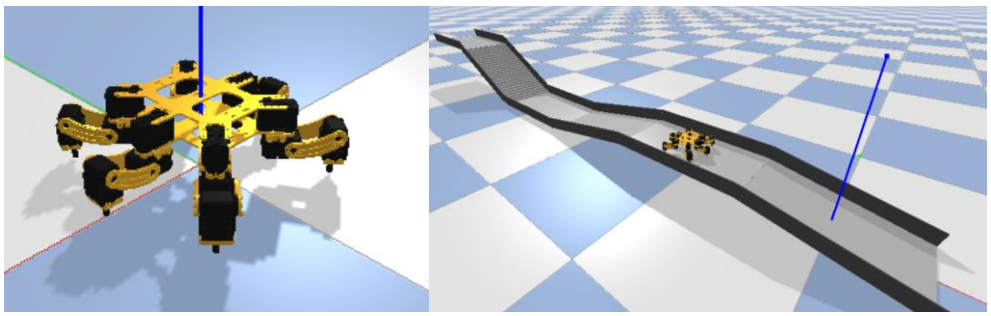
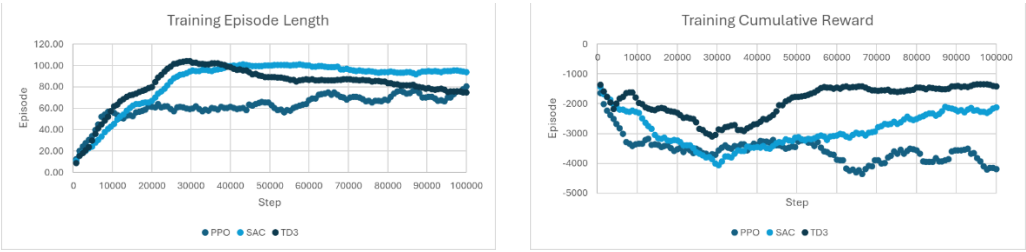


Fig 3: Training Environment

1. Training Performance and Stability:

The training curves in Fig. 3 illustrate the distinct learning dynamics of each algorithm. While TD3 achieved the highest final cumulative reward of approximately -1300, outperforming SAC (-2200) and PPO (-4400), this metric alone does not capture the full picture. A more critical indicator of robustness is the episode length, which reflects the policy's ability to avoid terminal states (e.g., falling over). As shown in Fig. 3(a), the SAC agent demonstrated superior stability, consistently achieving longer episodes (90-95 steps) in the final phase of training. In contrast, PPO exhibited initial promise but suffered a policy collapse, while TD3's performance was less consistent. This suggests that SAC, driven by its maximum entropy objective, learned a more cautious and resilient policy, prioritizing survival and stability over aggressive reward optimization. This inherent robustness is a highly desirable trait for real-world deployment.

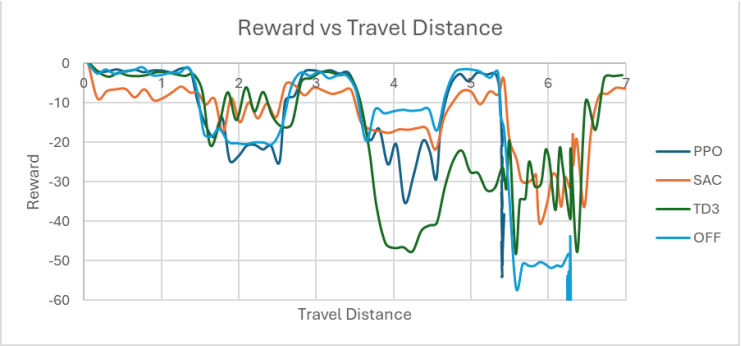


(a) (b)

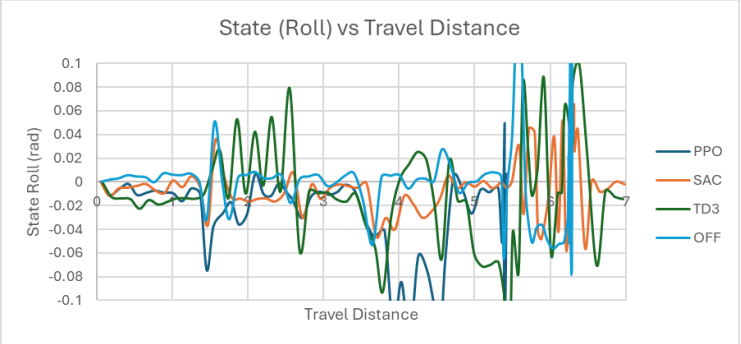
Fig 4: Training curves for PPO, SAC, and TD3.
(a) Episode Length vs. Training Steps.
(b) Cumulative Reward vs. Training Steps.

2. Testing Performance and Analysis:

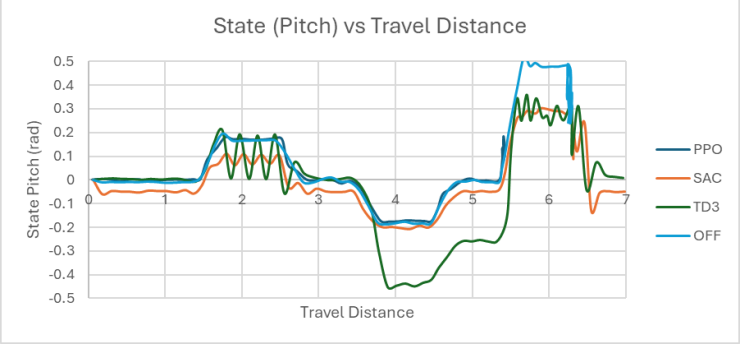
To assess the final learned policies, the trained models were tested on the full simulated obstacle course. The results, depicted in Fig. 4, confirmed SAC's superior performance.



(a)



(b)



(c)

Fig. 5: Testing curves for PPO, SAC, TD3, and without RL
(a) Reward (b) Roll (c) Pitch

- **PPO:** The PPO agent failed catastrophically at the 6.4-meter mark, which corresponded to the beginning of the stair's obstacle. As seen in Fig. 4(c), its policy commanded an extreme and unrecoverable pitch change, leading to immediate instability and termination. Its control signals for both roll and pitch were characterized by large, uncontrolled oscillations leading up to the failure point.
- **TD3:** The TD3 agent successfully completed the entire course. However, its control actions were visibly more aggressive and less smooth compared to SAC. The state graphs in Fig. 4(b) and 4(c) show that TD3's roll and pitch exhibited larger and more frequent oscillations, indicating a more reactive and less stable control strategy. While effective, this aggressive behavior could lead to higher mechanical stress and energy consumption on a physical robot.

- **SAC:** The SAC agent not only completed the course but did so with exceptional stability. Its policy maintained the robot's roll and pitch within a tight band around zero, with deviations occurring only when necessary to navigate obstacles. Its cumulative reward curve (Fig. 4(a)) is smoother and more consistent than the other algorithms. The final cost-reward analysis in Table I, which integrates penalties for instability and inefficiency over the entire run, quantitatively confirms SAC's superiority, with a final score of 22.53. This success is directly attributable to SAC's entropy maximization, which encourages the agent to explore a diverse range of smooth actions, ultimately converging on a policy that is both optimal and robust.

For further reference, Table 4 provides a concise summary of the three reinforcement learning algorithms.

Table 4: Comparative Analysis of PPO, TD3, and SAC

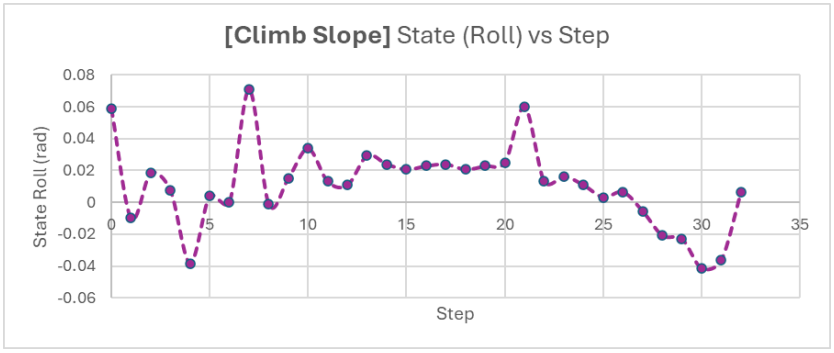
Algorithm	Equation Focus	Impact on Robot
PPO (Proximal Policy Optimization)	Constrains rapid changes in actions compared to the previous step	Stable under normal conditions, but less adaptive in extreme terrains
TD3 (Twin Delayed Deep Deterministic Policy Gradient)	Selects the best action directly while using two Q-value estimates for safety	Effective and fast, but results in more aggressive movements
SAC (Soft Actor-Critic)	Maximizes reward while encouraging action diversity (entropy)	Produces the smoothest, most stable, and most efficient movements

B. *Real-World Deployment and Validation*

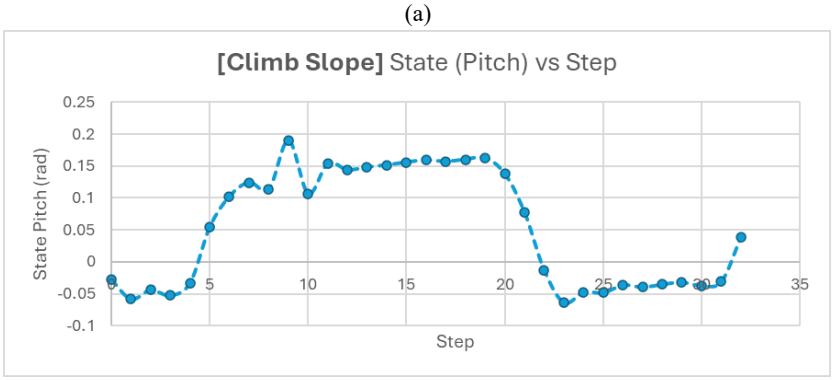
Based on its superior performance in simulation, the trained SAC model was deployed on the "BumbleBee" platform to assess its sim-to-real transferability. The robot was tested on three physical terrains: an incline, a decline, and a set of stairs. The results demonstrated a successful transfer of the learned policy, with the robot autonomously adapting its behavior in real time.

1. Incline Navigation:

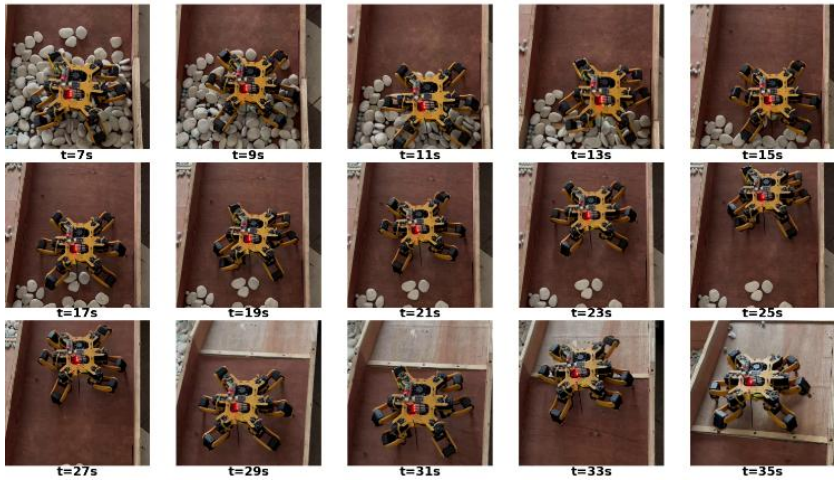
Upon encountering the incline, the policy demonstrated intelligent adaptation. In the initial phase (steps 0-5), the robot maintained a stable forward motion. As it began to climb (steps 5-22), the policy commanded a progressive pitch adjustment down to -0.08 radians to align the robot's body with the slope. This active postural control allowed the robot to successfully ascend the 3-meter incline in 32 steps.



(a)



(b)

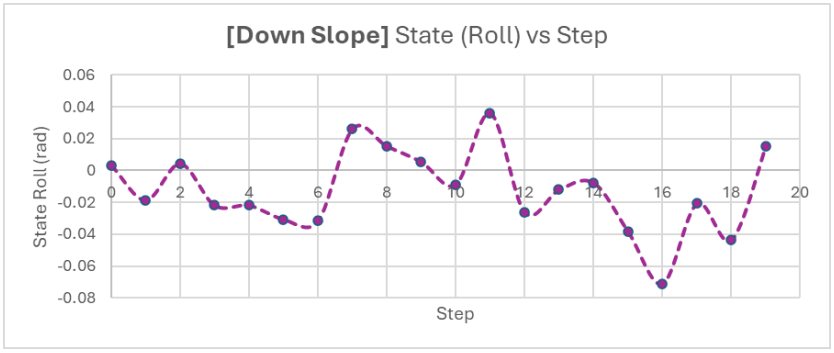


(c)

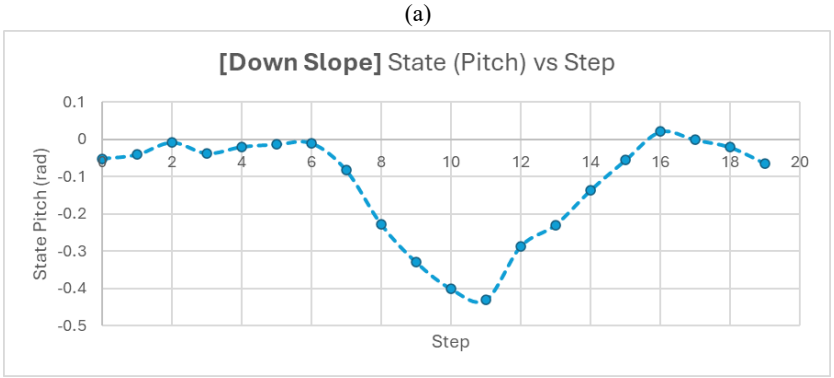
Fig. 6. Real-world performance on incline.
(a) Robot roll state (b) Robot pitch state (c) Robot step

2. Decline Navigation:

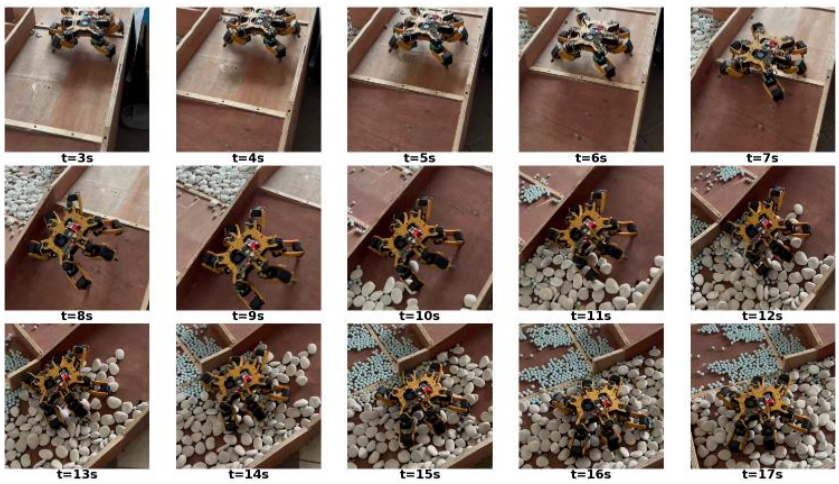
Navigating the decline, the robot completed the task efficiently in just 19 steps. The SAC policy exhibited a proactive "braking" strategy, reducing its forward velocity and adjusting its pitch to maintain stability during the descent. The body's pitch was modulated down to -0.25 radians at the steepest point, effectively preventing the robot from tipping forward. Similar to the incline test, the roll control was highly active, suggesting that the policy aggressively corrected for any lateral instability.



(a)



(b)



(c)

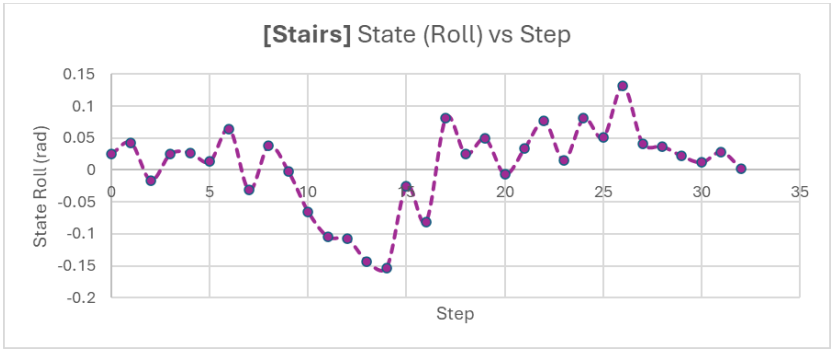
Fig. 7. Real-world performance on decline.
(a) Robot roll state (b) Robot pitch state (c) Robot step

3. Stairs Navigation:

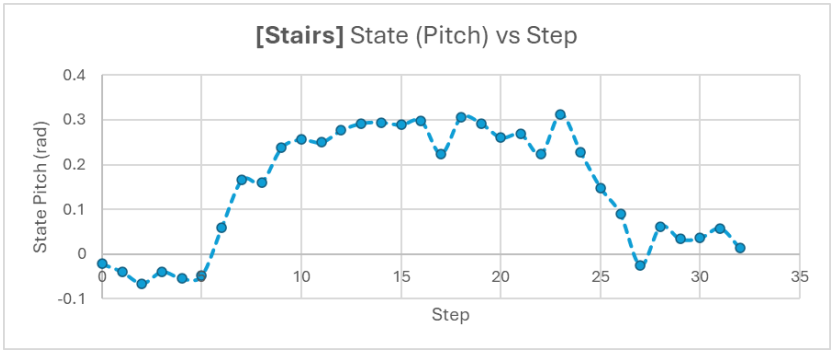
The stairs represented the most demanding test of the robot's adaptive capabilities. The SAC policy proved highly effective, enabling the robot to climb the entire staircase in 32 steps. This was achieved through a dynamic and multi-faceted adaptation:

- The robot consistently increased its forward pushing force (commanding a Body X position of -0.8 cm) in the initial steps to make contact with the first stair.

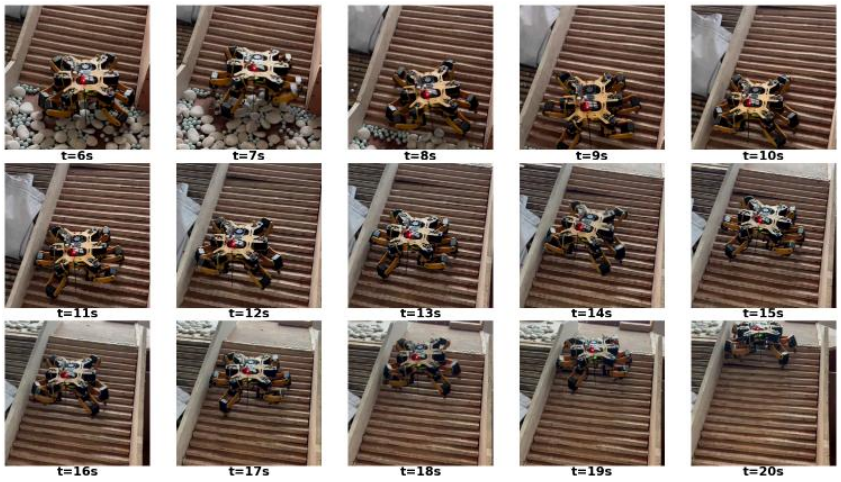
- During the most critical phase (steps 10-25), the policy commanded a maximum pitch angle of +0.4 radians at step 15, providing the necessary body lift to clear the height of each step (Fig. 5).
- Simultaneously, the roll control was highly active, with commands ranging from -1.0 to 0 radians, indicating an intense effort to maintain lateral balance as individual legs moved between steps.



(a)



(b)



(c)

Fig. 8. Real-world performance on stairs.
(a) Robot roll state (b) Robot pitch state (c) Robot step

While the robot successfully completed this challenging task, its motion was not perfectly smooth, with visible adjustments as it re-stabilized on each step.

This observation aligns with the findings from the simulation and suggests that the current reward function, while effective, could be improved by adding explicit penalties for jerk or high-frequency oscillations to encourage smoother locomotion on highly irregular surfaces. Nonetheless, the successful navigation of all three terrains with a 100% success rate and body stability maintained within ±0.3 radians validate the effectiveness of the integrated kinematic-RL approach for real-world applications.

5. Conclusion

This paper has presented a hierarchical control framework that successfully integrates classical kinematics with modern Reinforcement Learning to achieve adaptive locomotion for a hexapod robot. By leveraging a low-level kinematic controller, the high-level RL policy could learn complex behaviors in a simplified action space, enabling effective sim-to-real transfer.

Our comparative analysis of PPO, SAC, and TD3 demonstrated that Soft Actor-Critic (SAC) was the most suitable algorithm for this task, producing a policy that balanced performance with stability. When deployed on a physical robot, the SAC-trained model successfully navigated challenging real-world terrains, including inclines, declines, and stairs, with 100% reliability. The results confirm that this hybrid approach significantly enhances a hexapod robot's ability to autonomously adapt to complex and unstructured environments.

Future work will focus on three key areas. First, we will refine the reward function and conduct extensive hyperparameter tuning for the SAC algorithm to further improve motion smoothness and reduce oscillations. Second, we plan to enhance the robot's perceptual capabilities by integrating LiDAR or depth camera data into the observation space, allowing for explicit obstacle detection and proactive path planning. Finally, we will explore hardware optimizations, such as using lightweight carbon fiber materials and incorporating force sensors in the feet, to create a more robust and capable platform for real-world search and rescue missions.

Acknowledgment

I would like to thank Dr. Eng. Muhammad Abdillah, Dr. Eng. Novanto Yudistira, Dr. Arie Sukma Jaya, and Universitas Pertamina for their valuable support and guidance during this work.

References

[1] T. Deepa, S. Angalaeswari, D. Subbulekshmi, S. Krithiga, S. Sujeeth, and R. Kathiravan, “Design and implementation of bio inspired hexapod for exploration applications,” *Mater Today Proc*, vol. 37, no. Part 2, pp. 1603–1607, 2020, doi: 10.1016/j.matpr.2020.07.165.

[2] L. Zhang, D. Li, F. Yang, and C. Liu, “Development and attitude control of a Hexapod bionic-Robot,” *2016 IEEE International Conference on Robotics and Biomimetics, ROBIO 2016*, pp. 77–82, 2016, doi: 10.1109/ROBIO.2016.7866301.

[3] J. Coelho, F. Ribeiro, B. Dias, G. Lopes, and P. Flores, “Trends in the control of hexapod robots: A survey,” *Robotics*, vol. 10, no. 3, pp. 1–22, 2021, doi: 10.3390/robotics10030100.

[4] W. Ouyang, H. Chi, J. Pang, W. Liang, and Q. Ren, “Adaptive Locomotion Control of a Hexapod Robot via Bio-Inspired Learning,” *Front Neurorobot*, vol. 15, no. January, 2021, doi: 10.3389/fnbot.2021.627157.

[5] S. Muslimin and D. Istardi, “Inverse Kinematics Analysis for Motion Prediction of a Hexapod Robot,” *Proceedings of the 2018 International Conference on Applied Engineering, ICAE 2018*, pp. 1–5, 2018, doi: 10.1109/INCAE.2018.8579402.

[6] I. A. Kurniawan, Feriyonika, and S. Pramono, “Inverse dan Body Kinematics pada Robot Hexapod,” *Industrial Research Workshop and National Seminar*, vol. 9, pp. 115–123, 2018.

[7] F. Tedeschi and G. Carbone, “Hexapod walking robot locomotion,” *Mechanisms and Machine Science*, vol. 29, pp. 439–468, 2015, doi: 10.1007/978-3-319-14705-5_15.

[8] H. Xia, X. Zhang, and H. Zhang, “A new foot trajectory planning method for legged robots and its application in hexapod robots,” *Applied Sciences (Switzerland)*, vol. 11, no. 19, 2021, doi: 10.3390/app11199217.

[9] D. Li, W. Wei, and Z. Qiu, “Combined Reinforcement Learning and CPG Algorithm to Generate Terrain-Adaptive Gait of Hexapod Robots,” *Actuators*, vol. 12, no. 4, pp. 1–12, 2023, doi: 10.3390/act12040157.

[10] S. Ji, W. Wei, X. Liu, and J. Wu, “Optimization Control of Attitude Stability for Hexapod Robots Based on Reinforcement Learning,” *IEEE Access*, vol. 12, no. November, pp. 156363–156377, 2024, doi: 10.1109/ACCESS.2024.3485506.

[11] Q. Yang, Y. Gao, and S. Li, “Terrain-Adaptive Central Pattern Generators with Reinforcement Learning for Hexapod Locomotion,” *Proceedings - 2023 China Automation Congress, CAC 2023*, pp. 1017–1022, 2023, doi: 10.1109/CAC59555.2023.10451414.